

Caddy ≥2.11.0 ARI-refetch feasibility + the storage-surgery renewal path

Revision: 1 **Last modified:** 2026-07-01T11:40:00Z **Authority:** Inherits constitution/Constitution.md per §11.4.35. §11.4.150 deep-research deciding the LE Phase-5 renewal approach. Source-verified 2026-07-01.

Verdict — does bumping to Caddy ≥2.11.0 enable on-demand ARI re-fetch? NO-GO.

FACT (source-proven, certmagic v0.25.2 + acmez/v3 v3.1.6 = Caddy 2.11.0's pins): a network ARI re-fetch is triggered by exactly one condition, `cert.ari.NeedsRefresh()`, which in acmez/v3 v3.1.6 acme/ari.go returns true only when the cached `RetryAfter` has elapsed:

```
func (ari RenewalInfo) NeedsRefresh() bool {
    if !ari.HasWindow()      { return false }
    if ari.RetryAfter == nil { return true  }
    return time.Now().After(*ari.RetryAfter) // the SOLE gate
}
```

`updateARI` (the only function that hits the `renewalInfo` endpoint) has exactly two call sites (`config.go` `manageOne`, `maintain.go` `RenewManagedCertificates`), **both gated by `NeedsRefresh()`**. There is NO `POST /load`, `admin-API`, or `startup` path that bypasses the cached `RetryAfter`. Behaviour is identical to `certmagic v0.21.3`. **So the Caddy 2.11.0 bump does not move the decisive seam** — it is worth doing (fixes the #354 panic, RFC 9773 via `acmez/v3`, ARI `TryLock`) and *forces* the `libdns v1` migration anyway, but it is NOT the ARI-refresh unblocker it was hypothesised to be.

Caddy tag	certmagic	acmez	libdns
v2.10.0	v0.23.0	v3.1.2	v1.0.0-beta.1
v2.10.2	v0.24.0	v3.1.2	v1.1.0
v2.11.0	v0.25.2	v3.1.6	v1.1.1

v2.11.0 is the first release with certmagic $\geq$ v0.25.1. (The libdns-v1 migration is already required at Caddy $\geq$ 2.10.)

The path that WORKS — storage surgery on the CURRENT 2.8.4 stack (GO)

certmagic renews a cert when its **cached** ARI `_selectedTime` is in the past (it does NOT re-check `RetryAfter` in the renewal decision — only in the *fetch* gate). So rewriting the cert's cached ARI window to the past, then reloading it into memory, triggers a genuine zero-downtime ACME renewal — no version bump, no re-fetch.

Confirmed on-disk structure (§11.4.6, Caddy 2.8.4): `/data/caddy/certificates/pebble-14000-dir/<host>/<host>.json` →

```
{ "sans": [...], "issuer_data": { "url": ..., "ca": ...,
  "renewal_info": {
    "suggestedWindow": {"start": "...", "end": "..."},
    "_uniqueIdentifier": "...", "_retryAfter": "<+6h>",
    "_selectedTime": "<future>" } } }
```

The mechanism (proven live 2026-07-01):

1. Rewrite `issuer_data.renewal_info.suggestedWindow` + `_selectedTime` to the PAST (leave `_retryAfter` as-is so `NeedsRefresh()` stays false — no re-fetch overwrites the edit).
2. **Restart Caddy** — a config `/load` keeps the in-memory cert (with the old future window); only a restart re-loads `issuer_data` from storage into the in-memory cert.
3. Caddy's next maintenance tick reads the past `_selectedTime` → logs certificate needs renewal based on ARI window → renewing certificate → certificate renewed successfully. New serial + later `notBefore`; the renewal SWAP (old leaf → new leaf) is zero-downtime (Caddy serves the old leaf until the new one is ready). The restart is TEST SCAFFOLDING (a brief downtime *before* the cert is served); in production the maintenance loop fires on its own schedule with no restart at all.

Reference impl: `deploy/letsencrypt/phase5_rotation.sh` (re-runnable §11.4.98). Proven runs: S1→S2 with `swap_availability ok=N fails=0` and cert-analyzer PASS on S2 (chain to the per-run Pebble CA). Guard: `tests/letsencrypt/phase5_rotation_guard.sh` (RED = no-surgery → no renewal).

Alternatives (§11.4.6)

- **(a) storage surgery — CHOSEN** (above): deterministic, zero-downtime swap, works on 2.8.4.
- (b) wait out Retry-After (6h) — deterministic but too slow for a gate.
- (c) wipe /data → re-ISSUE (new serial, WITH downtime) — proves rotation, not zero-downtime renewal.
- On-demand reload-triggered re-fetch is *unimplemented in certmagic* (not structurally impossible) — would need an upstream contribution; tracked separately from this project.

libdns v0.2.x → v1.0.0 migration (for the independent Caddy-2.11 modernization)

Record is now an interface (RR() RR); field access → rec.RR(), value field Value→Data. AppendRecords/DeleteRecords signatures + libdns.AbsoluteName unchanged; certmagic's DNSProvider is still RecordAppender + RecordDeleter. The two provider methods become: rr := rec.RR(); host := fqdn(rr.Name, zone); ...{"value": rr.Data}. go.mod: caddy/v2 v2.11.0 + libdns/libdns v1.1.1; Dockerfile.caddy ARG CADDY_VERSION=2.11.0. UNCONFIRMED: existence of the caddy:2.11.0-{builder,alpine} tags — the build's list-modules gate proves it at build time.

Sources verified 2026-07-01

Caddy go.mod pins v2.11.0/v2.10.2/v2.10.0
(raw.githubusercontent.com) · certmagic v0.25.2 config.go/
maintain.go/certificates.go/solvers.go · acmez/v3 v3.1.6 acme/
ari.go · libdns v1.0.0 record.go/rrtypes.go · caddyserver/caddy
issues #6789/#6943 · caddy.community "force renewal" + "ari
renew on startup" threads · RFC 9773 · project prior docs/
research/pebble_set_renewal_info_20260701/ANALYSIS.md.